

Overview of Perceptron

Subject: Perceptron

$$y = \text{sign}(w \cdot x + b)$$

$$w \leftarrow w + \eta(y - \hat{y})x$$

1.

$d_j \in \{0, 1\}$ is the desired output value of the perceptron for that input. We show the values of the features as follows:

2.

$D = \{(\mathbf{x}_1, d_1), \dots, (\mathbf{x}_s, d_s)\}$ is the training set of s samples, where:

3.

The artificial neuron and artificial neural network were invented in 1943 by Warren McCulloch and Walter Pitts in their seminal paper "A Logical Calculus of the Ideas Immanent in Nervous Activity". In 1957, Frank Rosenblatt was at the Cornell Aeronautical Laboratory. He simulated the perceptron on an IBM 704. Later, he obtained funding by the Information Systems Branch of the United States Office of Naval Research and the Rome Air Development Center, to build a custom-made computer, the Mark I Perceptron. It was first publicly demonstrated on 23 June 1960. The machine was "part of a previously secret four-year NPIC [the US' National Photographic Interpretation Center] effort from 1963 through 1966 to develop this algorithm into a useful tool for photo-interpreters". Rosenblatt described the details of the perceptron in a 1958 paper. His organization of a perceptron is constructed of three kinds of cells ("units"): S, A, R, which stand for "sensory", "association" and "response". He presented at the first international symposium on AI, Mechanisation of Thought Processes, which took place in 1958 November. Rosenblatt's project was funded under Contract Nonr-401(40) "Cognitive Systems Research Program", which lasted from 1959 to 1970, and Contract Nonr-2381(00) "Project PARA" ("PARA" means "Perceiving and Recognition Automata"), which lasted from 1957 to 1963. In 1959, the Institute for Defense Analysis awarded his group a \$10,000 contract. By September 1961, the ONR awarded further \$153,000 worth of contracts, with \$108,000 committed for 1962. The ONR research manager, Marvin Denicoff, stated that ONR, instead of ARPA, funded the Perceptron project, because the project was unlikely to produce technological results in the near or medium term. Funding from ARPA go up to the order of millions dollars, while from ONR are on the order of 10,000 dollars. Meanwhile, the head of IPTO at ARPA, J.C.R. Licklider, was interested in 'self-organizing', 'adaptive' and other biologically-inspired methods in the 1950s; but by the mid-1960s he was openly critical of these, including the perceptron. Instead he strongly favored the logical AI approach of Simon and Newell.

4.

Further reading Aizerman, M. A. and Braverman, E. M. and Lev I. Rozonoer. Theoretical foundations of the potential function method in pattern recognition learning. Automation and Remote Control, 25:821–837, 1964. Rosenblatt, Frank (1958), The Perceptron: A Probabilistic Model for Information Storage and Organization in the Brain, Cornell Aeronautical Laboratory, Psychological Review, v65, No. 6, pp. 386–408. doi:10.1037/h0042519. Rosenblatt, Frank (1962), Principles of Neurodynamics. Washington, DC: Spartan Books. Minsky, M. L. and Papert, S. A. 1969. Perceptrons. Cambridge, MA: MIT Press. Gallant, S. I. (1990). Perceptron-based learning algorithms. IEEE Transactions on Neural Networks, vol. 1, no. 2, pp. 179–191. Olazaran Rodriguez, Jose Miguel. A historical sociology of neural network research. PhD Dissertation. University of Edinburgh, 1991. Mohri, Mehryar and Rostamizadeh, Afshin (2013). Perceptron Mistake Bounds arXiv:1305.0208, 2013. Novikoff, A. B. (1962). On convergence proofs on perceptrons. Symposium on the Mathematical Theory of Automata, 12, 615–622. Polytechnic Institute of Brooklyn. Widrow, B., Lehr, M.A., "30 years of Adaptive Neural Networks: Perceptron, Madaline, and Backpropagation," Proc. IEEE, vol 78, no 9, pp. 1415–1442, (1990). Collins, M. 2002. Discriminative training methods for hidden Markov models: Theory and experiments with the perceptron algorithm in Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP '02). Yin, Hongfeng (1996), Perceptron-Based Algorithms and Analysis, Spectrum Library, Concordia University, Canada

5.

A single perceptron is a linear classifier. It can only reach a stable state if all input vectors are classified correctly. In case the training set D is not linearly separable, i.e. if the positive examples cannot be separated from the negative examples by a hyperplane, then the algorithm would not converge since there is no solution. Hence, if linear separability of the training set is not known a priori, one of the training variants below should be used. Detailed analysis and extensions to the convergence theorem are in Chapter 11 of Perceptrons (1969). Linear separability is testable in time $\min(O(n^{d/2}), O(d^{2n}), O(n^{d-1} \ln n))$, where n is the number of data points, and d is the dimension of each point. If the training set is linearly separable, then the perceptron is guaranteed to converge after making finitely many mistakes. The theorem is proved by Rosenblatt et al.

6.

The following simple proof is due to Novikoff (1962). The idea of the proof is that the weight vector is always adjusted by a bounded amount in a direction with which it has a negative dot product, and thus can be bounded above by $O(\sqrt{t})$, where t is the number of changes to the weight vector. However, it can also be bounded below by $O(t)$ because if there exists an (unknown) satisfactory weight vector, then every change makes progress in this (unknown) direction by a positive amount that depends only on the input vector.

7.

Learning a Boolean function Consider a dataset where the x are from $\{-1, +1\}^n$, that is, the vertices of an n -dimensional hypercube centered at origin, and $y = \theta(x_i)$. That is, all data points with positive x_i have $y = 1$, and vice versa. By the perceptron convergence theorem, a perceptron would converge after making at most n mistakes. If we were to write a

logical program to perform the same task, each positive example shows that one of the coordinates is the right one, and each negative example shows that its complement is a positive example. By collecting all the known positive examples, we eventually eliminate all but one coordinate, at which point the dataset is learned. This bound is asymptotically tight in terms of the worst-case. In the worst-case, the first presented example is entirely new, and gives n bits of information, but each subsequent example would differ minimally from previous examples, and gives 1 bit each. After $n + 1$ examples, there are $2n$ bits of information, which is sufficient for the perceptron (with $2n$ bits of information). However, it is not tight in terms of expectation if the examples are presented uniformly at random, since the first would give n bits, the second $n/2$ bits, and so on, taking $O(\ln n)$ examples in total.